



시각장애인을 위한 지능형 도보 내비게이션

8팀 눈이돼조

김지홍 윤수현 조준호 최은수



CONTENT



1. 주제 및 소개
2. 시연 영상 & 결과 분석
3. 핵심 기술 & 분석
4. 향후 연구 과제
5. 프로젝트 수행 후기



Project Overview



주제

- 시각장애인의 안전 보행을 위한 엣지 AI 기반 음성 안내 가이드

목표

- YOLO 모델을 활용한 실시간 다중 객체 분류 및 탐지
- Jetson 보드 내 Ollama(로컬 LLM) 구축 및 Ollama API와 파이썬 연동
- 오디오 출력 지원



결과

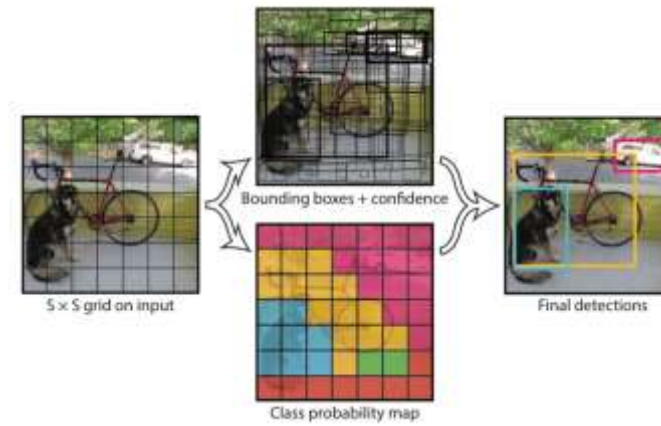
- 다중 YOLO 모델을 사용하여 장애물(ex. 사람, 개, 신호등, 교통수단, 횡단보도, 계단 등)에 대해 높은 신뢰성의 실시간 분류 및 탐지
- 분류 및 탐지된 장애물의 위치 정보에 따른 상황 맞춤형 안내 문자 생성
- 유무선(Bluetooth/USB) 스피커를 활용한 실시간 음성 출력



Project Overview



Jetson Orin Nano



YOLO



Cam



Ollama(LLM)

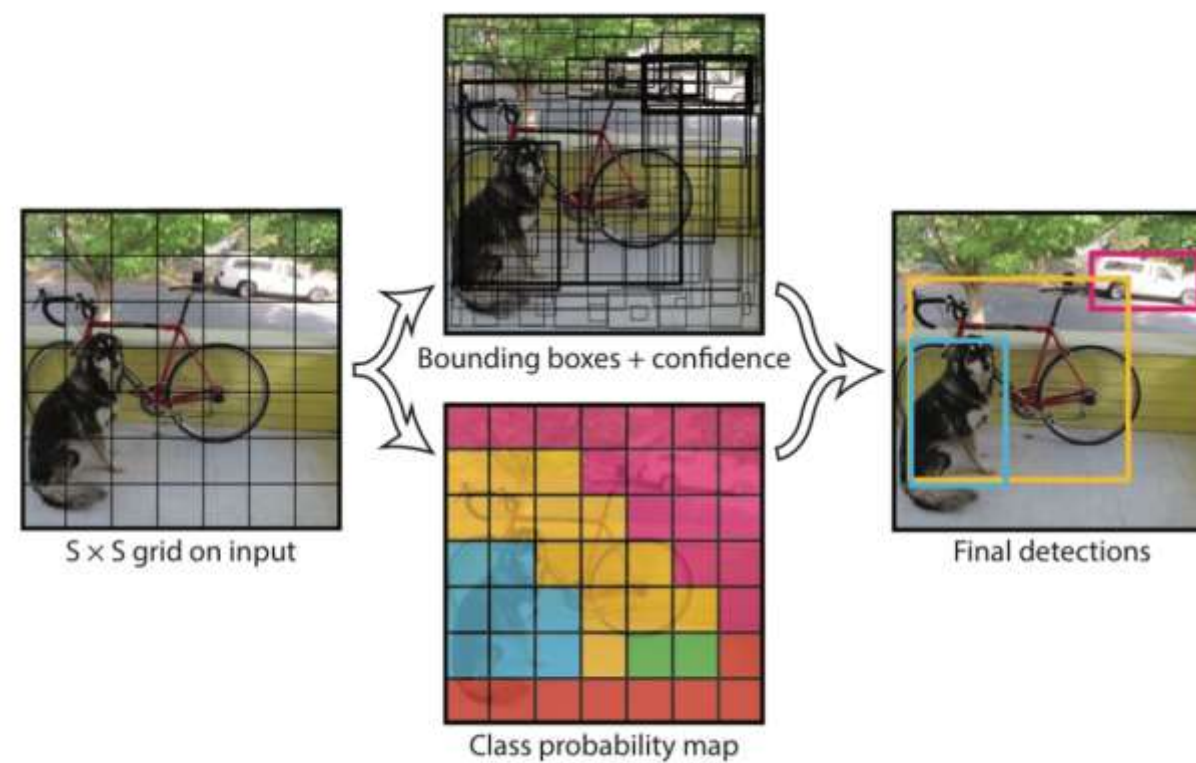


Speaker(Bluetooth, USB)



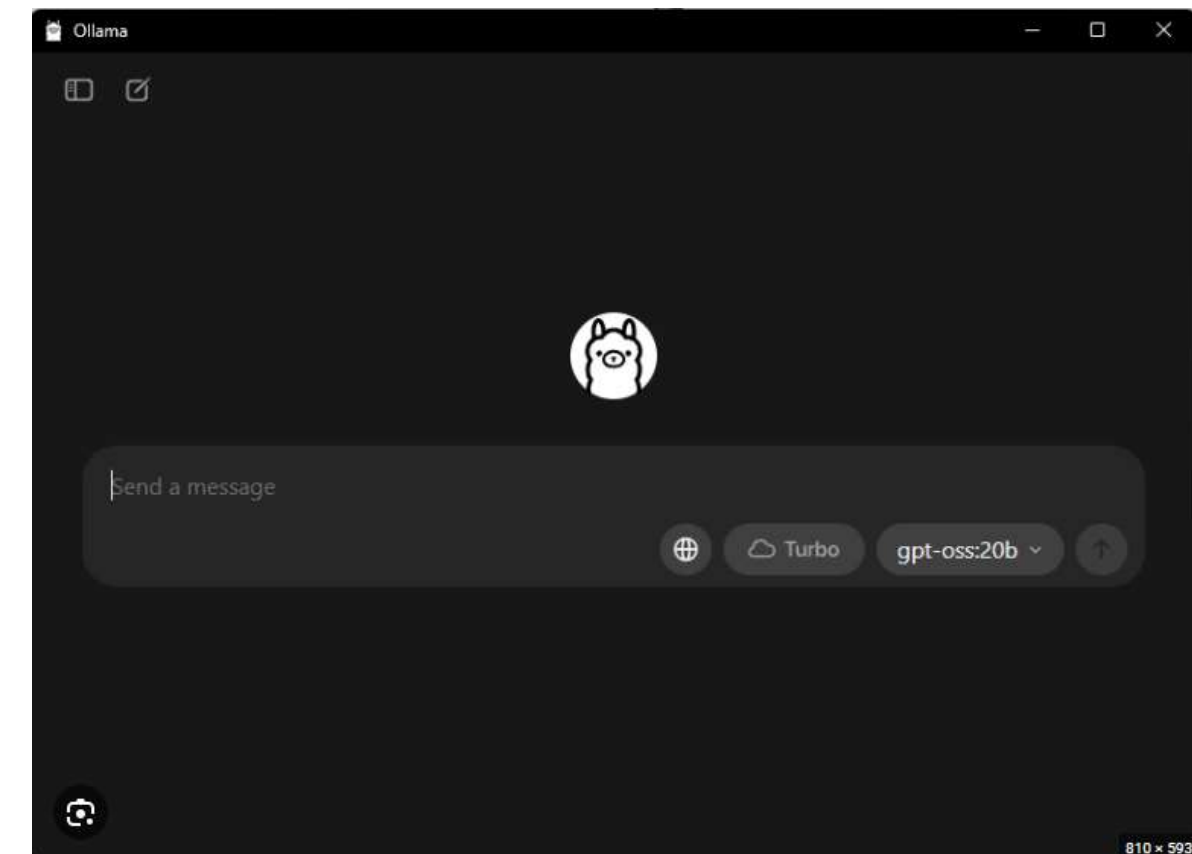
Project Overview

YOLO



- You Only Look Once
- 이미지 전체를 보고 사물을 찾아내는 초고속 실시간 객체 탐지 인공지능 모델

Ollama



- 대규모 AI 언어 모델(LLM)을 로컬 기기에서 무료로 다운로드하고 구동할 수 있게 해주는 오픈소스 인프라 플랫폼



Project Overview

Hardware Specification

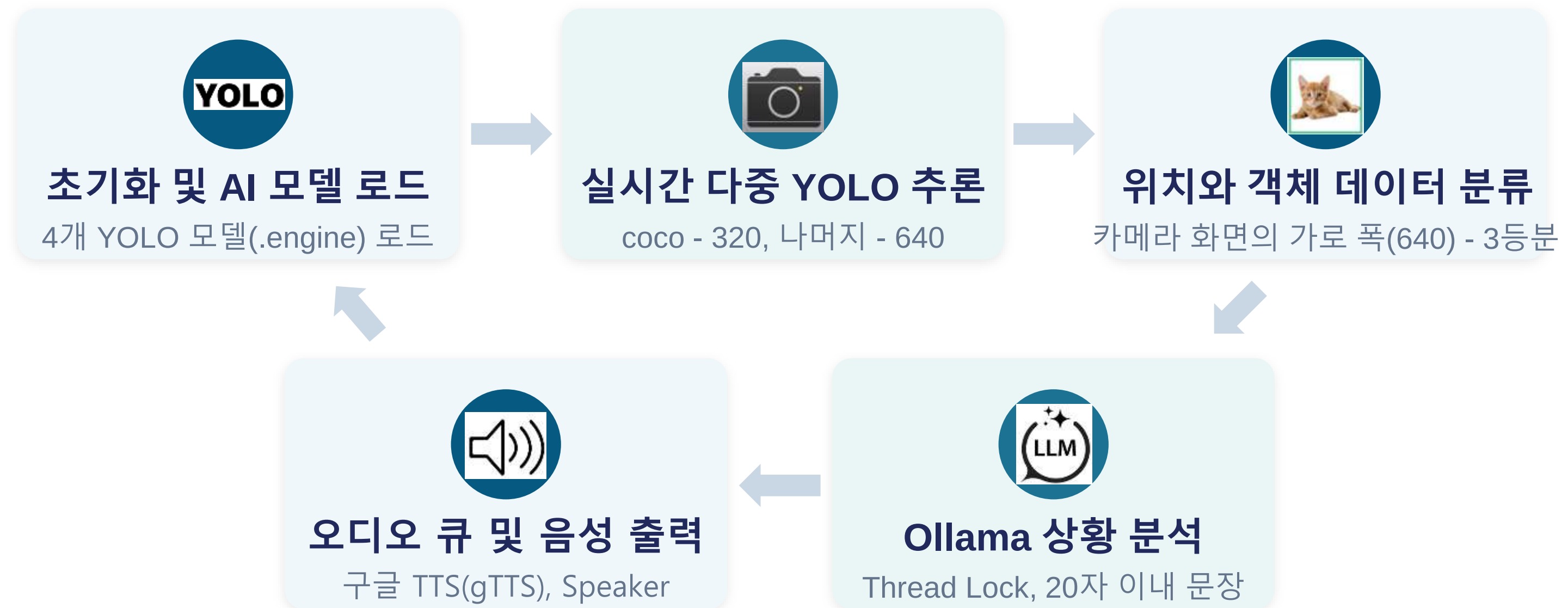
분류	항목	주요 사양	프로젝트 내 역할
Main Board	Jetson Orin Nano Developer Kit	NVIDIA Ampere architecture GPU, 블루투스 내장	온디바이스(On-Device) 실시간 다중 YOLO 추론 및 로컬, LLM(Ollama) 구동 핵심 연산 장치
Camera	Logitech C270 HD Webcam	HD 720p (최대 1280x720) 지원 프로젝트 설정: 640x480 pixels	실시간 보행 환경 영상 캡처 및 AI 모델 프레임 공급
Audio	Bluetooth 스피커, USB 스피커	-	시각장애인 대상 실시간 상황 안내 음성(gTTS) 출력

Software Specification

분류	항목	주요 사양
Language	Python	-
AI Framework	Ultralytics YOLO11	TensorRT (.engine) 포맷 변환 사용
AI Models	다중 Object Detection	• <code>coco_model</code> (imgsz=320) • <code>custom_model_1</code> (imgsz=640) • <code>custom_model_2</code> (imgsz=640) • <code>custom_model_3</code> (imgsz=640)
LLM Engine	Ollama	Gemma3:4b (로컬 경량화 LLM) -> Qwen2:0.5b
TTS Engine	gTTS (Google TTS)	온라인 API 기반 한국어 지원



Project Overview



※ Thread Lock : Ollama의 상황 분석이 끝날 때까지,
새로운 카메라 데이터가 LLM으로 유입되지 않도록
차단하는 안전장치

Video & Result

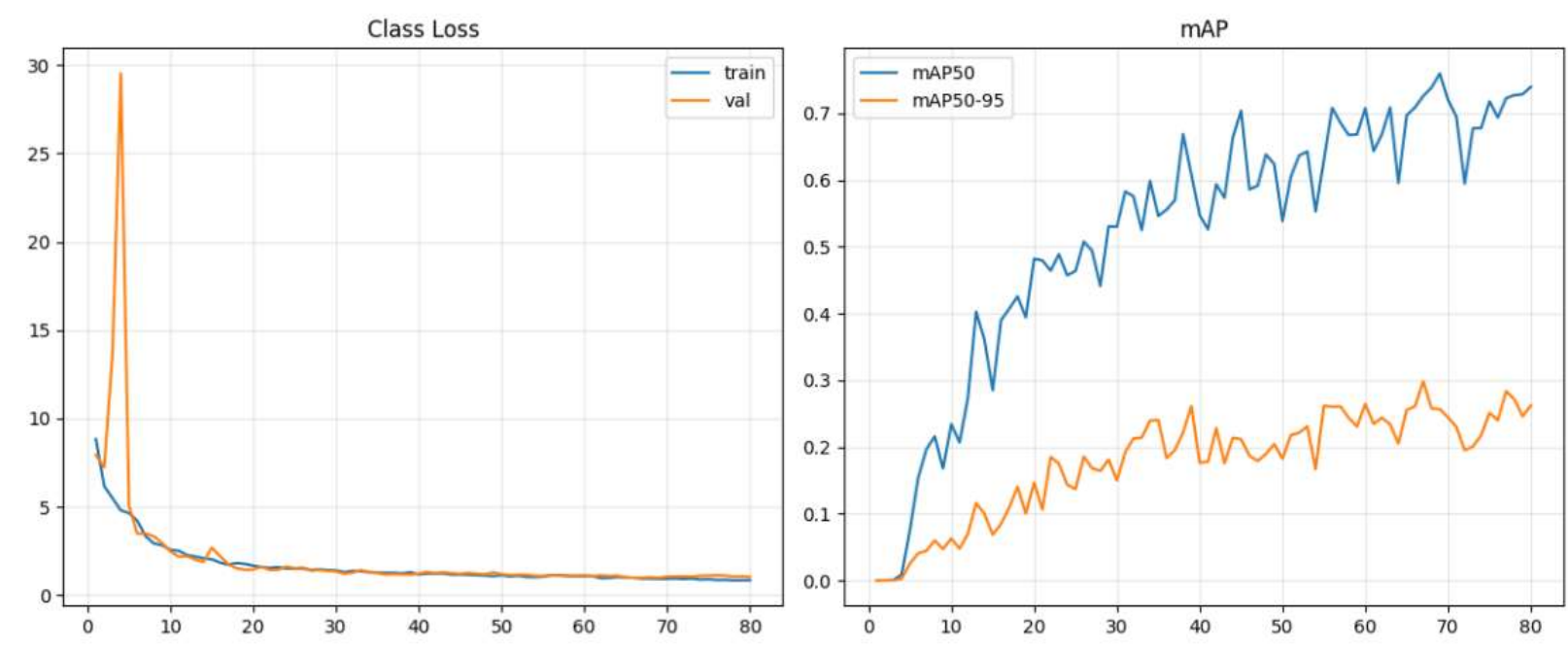


Video & Result



데이터 처리

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
76/80	2.39G	1.671	0.866	0.8146	9	640: 100%	31/31	3.4it/s	9.2s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	4/4	2.8it/s 1.4s
	all	108	182	0.807	0.636	0.693	0.24		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
77/80	2.39G	1.581	0.8759	0.8304	10	640: 100%	31/31	3.3it/s	9.3s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	4/4	2.8it/s 1.4s
	all	108	182	0.829	0.667	0.722	0.284		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
78/80	2.39G	1.616	0.8432	0.8152	7	640: 100%	31/31	3.7it/s	8.3s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	4/4	2.0it/s 2.0s
	all	108	182	0.839	0.663	0.727	0.271		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
79/80	2.39G	1.576	0.8535	0.829	8	640: 100%	31/31	4.0it/s	7.7s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	4/4	2.8it/s 1.4s
	all	108	182	0.837	0.637	0.728	0.246		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
80/80	2.39G	1.609	0.852	0.8057	9	640: 100%	31/31	3.4it/s	9.1s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	4/4	3.1it/s 1.3s
	all	108	182	0.848	0.679	0.74	0.262		



Video & Result

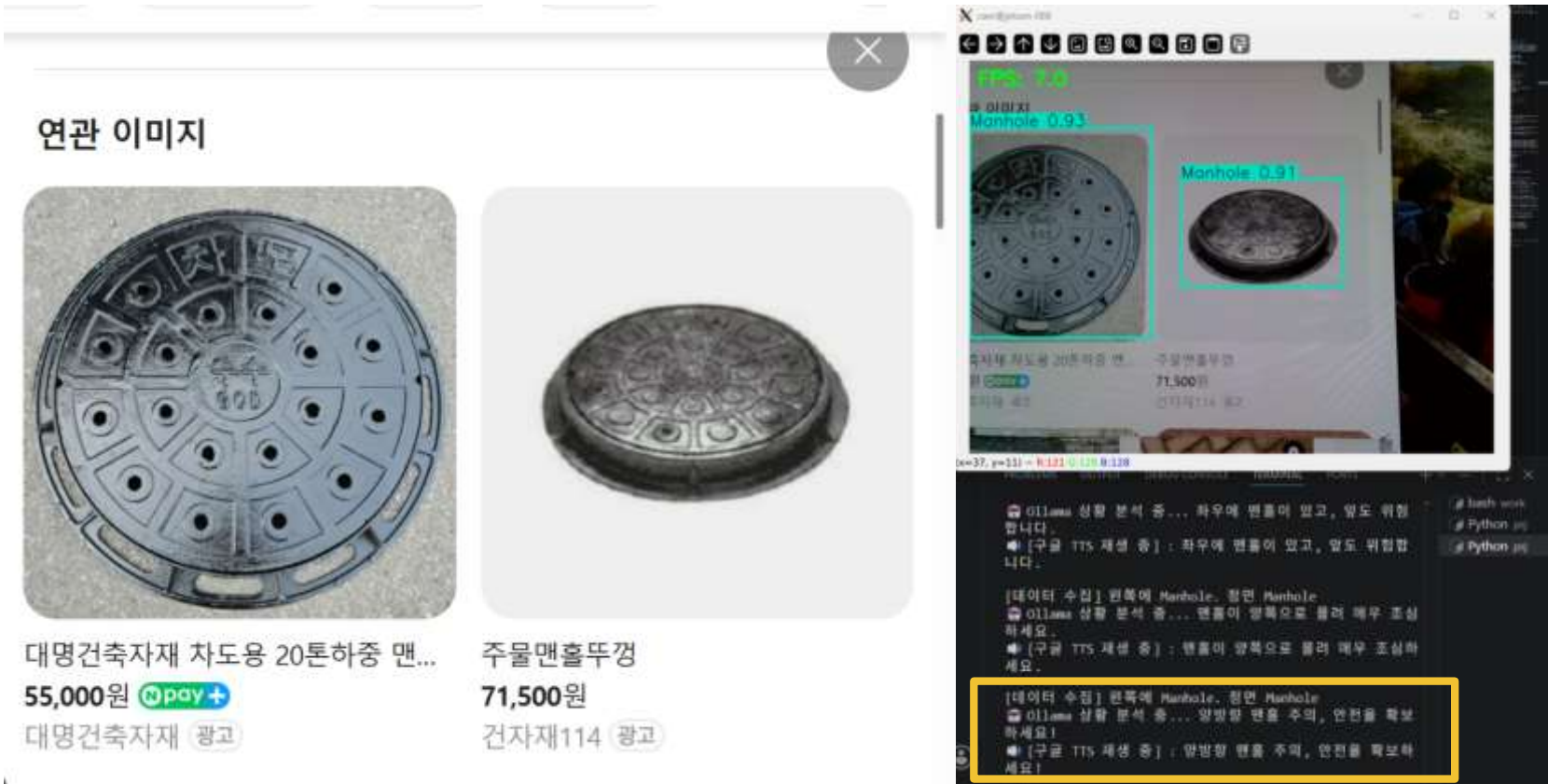


Real-Detection



Station Shelter

- 출력 데이터(원시): 오른쪽에 StationShelter
- 출력 데이터(Ollama): 오른쪽으로 역이 있으니 주의하세요.



Manhole

- 출력 데이터(원시): 왼쪽에 Manhole, 정면 Manhole
- 출력 데이터(Ollama): 양방향 맨홀 주의, 안전을 확보하세요!

Video & Result



Real-Detection



Rainy Night

- 출력 데이터(원시): 왼쪽에 빨간불
- 출력 데이터(Ollama): 왼쪽 빨간불, 정지해야 합니다

Video & Result



Real-Detection



Evening Downtown

- 출력 데이터(원시): 왼쪽에 사람, 정면 사람, greenlight, 오른쪽에 빨간불, 자동차
- 출력 데이터(Ollama): 사람, 녹색 신호, 차량에 주의하세요

Video & Result



Demo Video



Video & Result



Real-Time Detection 설정 필요 사항

```
jihong@jetson-008: ~  
jihong@jetson-008:~$ ollama serve  
time=2026-06-26T09:24:49.808+09:00 level=INF  
GML_VK_VISIBLE_DEVICES: GPU_DEVICE_ORDINAL:  
A_ARG_FIT: LLAMA_ARG_FIT_TARGET: NO_PROXY: O  
AMA_EDITOR: OLLAMA_FLASH_ATTENTION:false OLL
```

Ollama On



USB Speaker Connect(포트 연결)

```
jihong@jetson-008: ~  
jihong@jetson-008:~$ bluetoothctl connect 11:F0:10:DA:42:3E  
Attempting to connect to 11:F0:10:DA:42:3E  
[CHG] Device 11:F0:10:DA:42:3E Connected: yes  
[CHG] Device 11:F0:10:DA:42:3E Paired: yes  
Connection successful
```

Bluetooth Speaker Connect

Video & Result



아쉬운 점

- 1. Ollama 추론 지연 속도
- 2. 사물 오인식

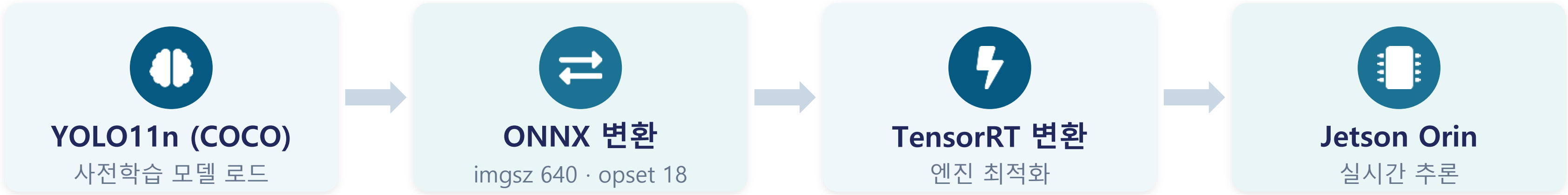


Skill & Analysis



Skill & Analysis

Data Processing



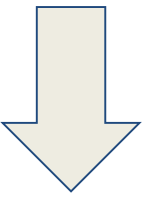
Skill & Analysis

Data Processing



```
'traffic_light',  
'traffic_sign_restriction',  
'traffic_sign_warning',  
'traffic_sign_instruction'
```

- 총 4개의 class로 분류



- class가 너무 세밀하여 민감하게 데이터가 잡히고 필요 없는 것들까지 인식을 함.

```
all_images = [f for f in os.listdir(train_img_dir) if f.lower().endswith(('.jpg', '.jpeg', '.png'))]  
print("이미지 파일 개수(json 제외):", len(all_images))  
  
val_size = int(len(all_images) * 0.2)  
val_images = random.sample(all_images, val_size)
```

- train 데이터에서 직접 20%의 데이터를 추출하여 val을 구성

Skill & Analysis

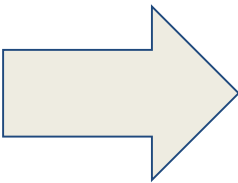
Data Processing



```
CLASSES = [  
    'traffic_light_red',  
    'traffic_light_yellow',  
    'traffic_light_green'  
]
```

```
def light_color(obj):  
    attr = obj.get('attribute', [])  
    if not attr:  
        return None  
    a = attr[0]  
  
    if a.get('red') == 'on':  
        return 'traffic_light_red'  
    if a.get('yellow') == 'on':  
        return 'traffic_light_yellow'  
    if a.get('green') == 'on':  
        return 'traffic_light_green'  
    return None
```

```
{  
    "light_count": "3",  
    "box": [  
        699,  
        63,  
        750,  
        82  
    ],  
    "attribute": [  
        {  
            "red": "off",  
            "green": "on",  
            "yellow": "off"  
        }  
    ],  
    "class": "traffic_light"  
},
```



Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	77/77 2.3it/s 33.6s		
1/80	1.33G	3.239	39.27	0.8262	2	320	100%	13/13 1.3it/s 10.1s	
	Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%		
	all	804	278	9.44e-06	0.00625	2.16e-07	1e-07		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	77/77 3.0it/s 25.3s		
2/80	1.34G	3.959	21.36	0.9218	3	320	100%	13/13 3.1it/s 4.2s	
	Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%		
	all	804	278	0.00228	0.0444	0.00334	0.00114		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	77/77 3.0it/s 25.7s		
3/80	1.34G	4.104	15.07	0.8976	5	320	100%	13/13 2.9it/s 4.6s	
	Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%		
	all	804	278	0.71	0.00625	0.00166	0.000531		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	77/77 2.9it/s 26.4s		
4/80	1.34G	4.247	9.69	0.9389	15	320	100%	13/13 3.2it/s 4.1s	
	Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%		
	all	804	278	0.703	0.00833	0.005	0.00115		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	77/77 3.0it/s 26.0s		
5/80	1.34G	4.35	6.43	0.93	11	320	100%	13/13 3.5it/s 3.8s	
	Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%		
	all	804	278	0.35	0.00208	0.00107	0.000292		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	77/77 2.9it/s 26.1s		
6/80	1.34G	4.163	5.19	0.9543	2	320	100%	13/13 2.3it/s 5.7s	
	Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%		
	all	804	278	0.712	0.0187	0.0146	0.00378		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	77/77 3.2it/s 24.4s		
7/80	1.34G	4.134	4.742	0.9246	7	320	100%	13/13 2.1it/s 6.1s	
	Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%		
	all	804	278	0.683	0.0104	0.00357	0.000733		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	31/77 3.6it/s 9.1s<12.7s		
8/80	1.34G	4.125	4.572	0.8969	13	320	40%		

JSON 라벨 → YOLO 형식 변환 후 1차 학습

- traffic_light의 attribute(red/green/yellow)를 색상별 클래스로 분리하고, 변환된 데이터로 YOLO 학습을 진행

Skill & Analysis

Data Processing

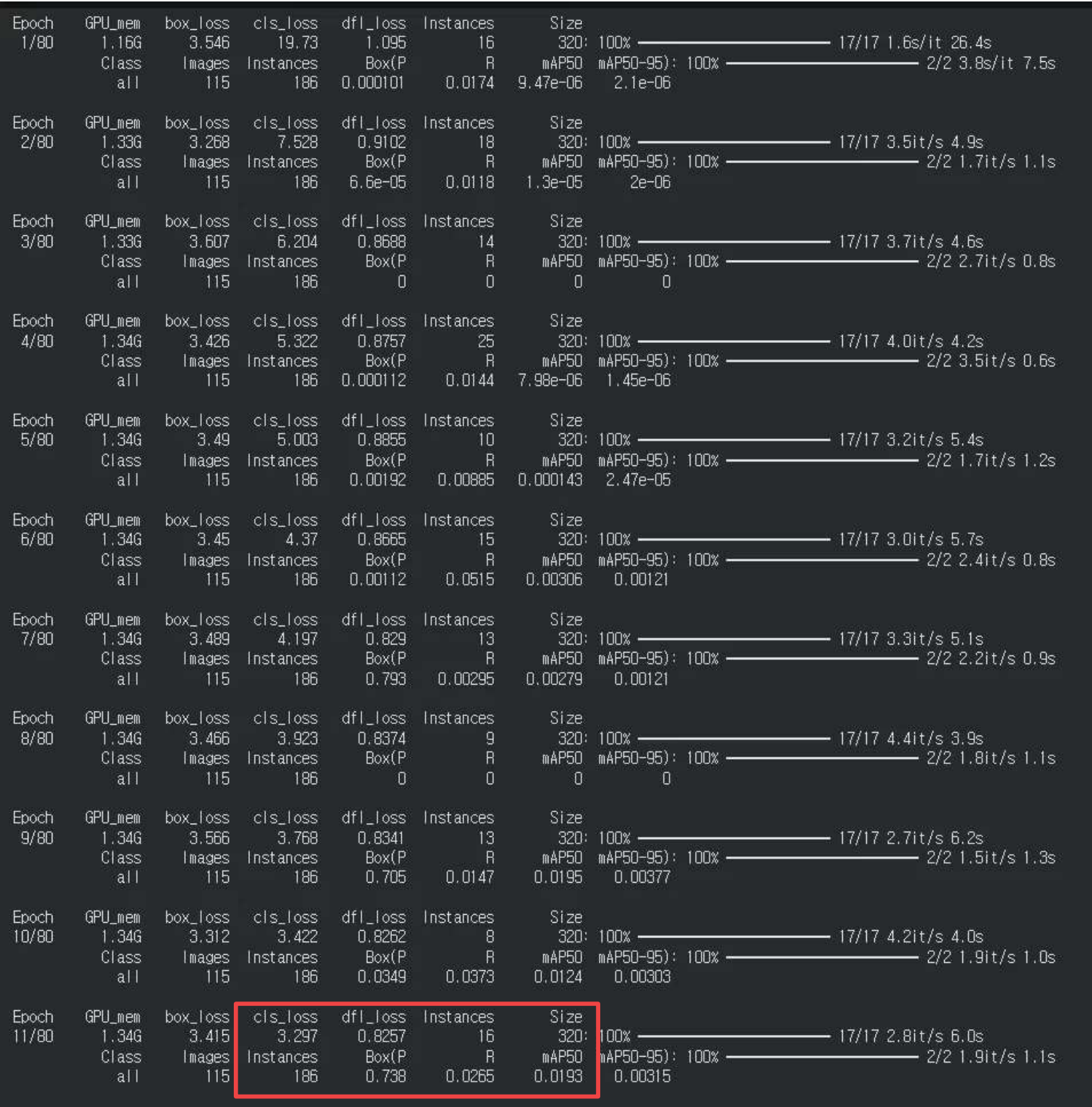


type	라벨 있음	라벨 없음	전체	라벨 있음 비율
train	441	2015	2456	18%
val	128	486	614	21%

- 데이터 중 traffic_light 에 대한 데이터의 비중이 너무 작음

```
def get_class_key(obj):  
    cls = obj.get('class')  
    if cls == 'traffic_light':  
        return get_light_color(obj)  
    return None
```

- class에서 traffic_sign은 제외



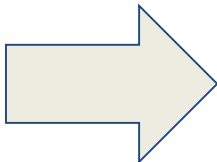
Skill & Analysis

Data Processing



```
현재 val 클래스 ID 분포: {'2': 113, '0': 60, '1': 13}
val 라벨 파일 수: 115

w 최소/최대/평균: 0.0063 / 0.1008 / 0.0197
h 최소/최대/평균: 0.0056 / 0.1583 / 0.0177
```



- val 분포가 한쪽으로 치우쳐 있음
- box가 너무 작아 못 잡음
- imgsz = 640으로 변경

```
DATA = f'{new_root}/data.yaml'

yolo = YOLO('yolo11n.pt')
yolo.train(data=DATA, epochs=80, batch=16,
           imgsz=640, device=0, workers=4,
           project=project_root,
           degrees=15.0, flipud=0.0, fliplr=0.5, mosaic=1.0,
           name='traffic_light_balanced_v3')
```

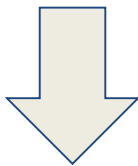
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size		
48/80	5.62G	2.045	1.21	0.8107	13	640: 100%	33/33	3.9it/s 8.4s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	4/4 2.3it/s 1.8s
	all	115	186	0.528	0.561	0.537	0.199	
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size		
49/80	5.62G	1.969	1.166	0.809	35	640: 100%	33/33	3.3it/s 10.0s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	4/4 2.6it/s 1.5s
	all	115	186	0.632	0.489	0.505	0.189	
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size		
50/80	5.62G	1.933	1.157	0.8201	25	640: 100%	33/33	3.1it/s 10.5s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	4/4 2.8it/s 1.4s
	all	115	186	0.814	0.43	0.487	0.201	

Skill & Analysis

Data Processing

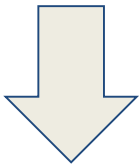


- 1. 노란색의 데이터가 부족함
- 1. 보행자 신호등에서의 노란색은 필요 없음



클래스별 객체 수:
traffic_light_red: 309
traffic_light_green: 569

- red 와 green의 객체 수가 다름
- 적은 red & green 데이터로만 학습시키면 과적합, 오인식 위험
-> label이 없는 데이터도 같이 학습



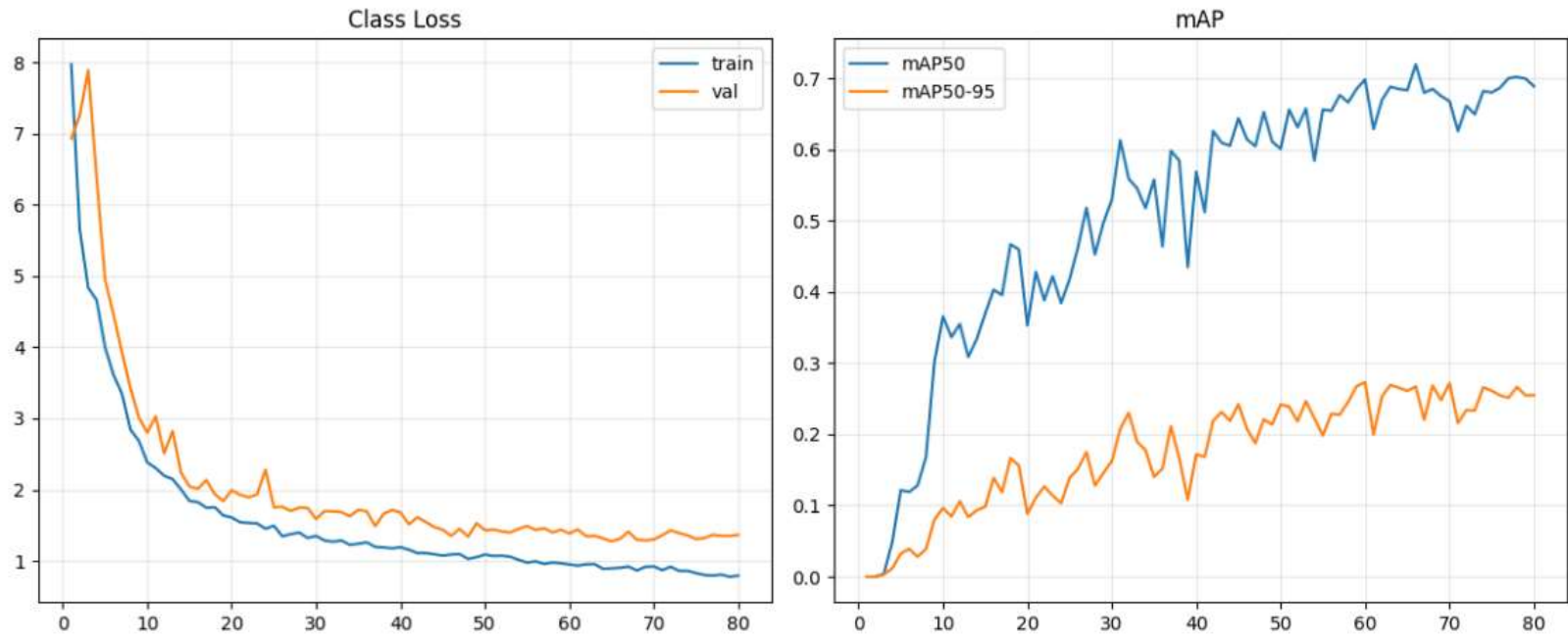
train 0.0 val 0.0 균형 안 맞춤	train 0.2 val 0.0 균형 안 맞춤	train 0.0 val 0.0 균형 맞춤	train 0.2 val 0.0 균형 맞춤
---	---	---	---

Skill & Analysis

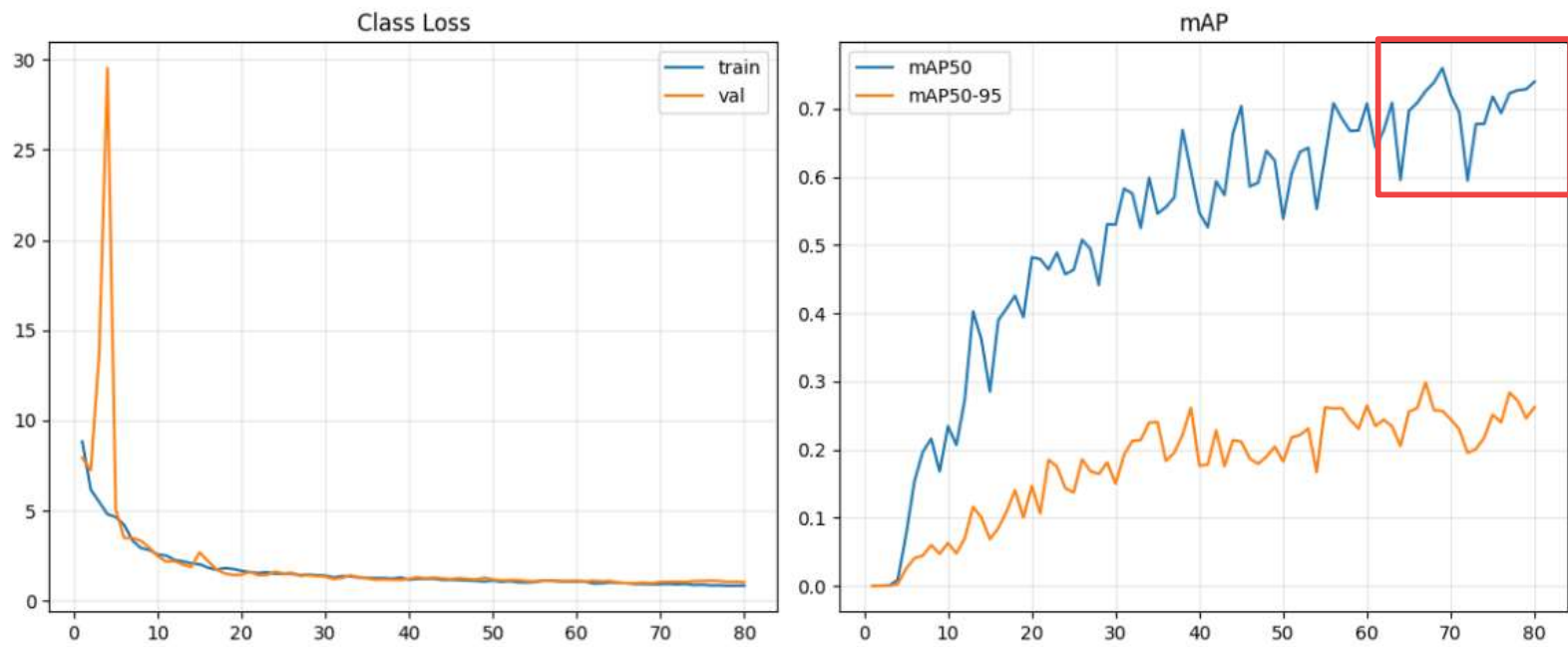
Data Processing



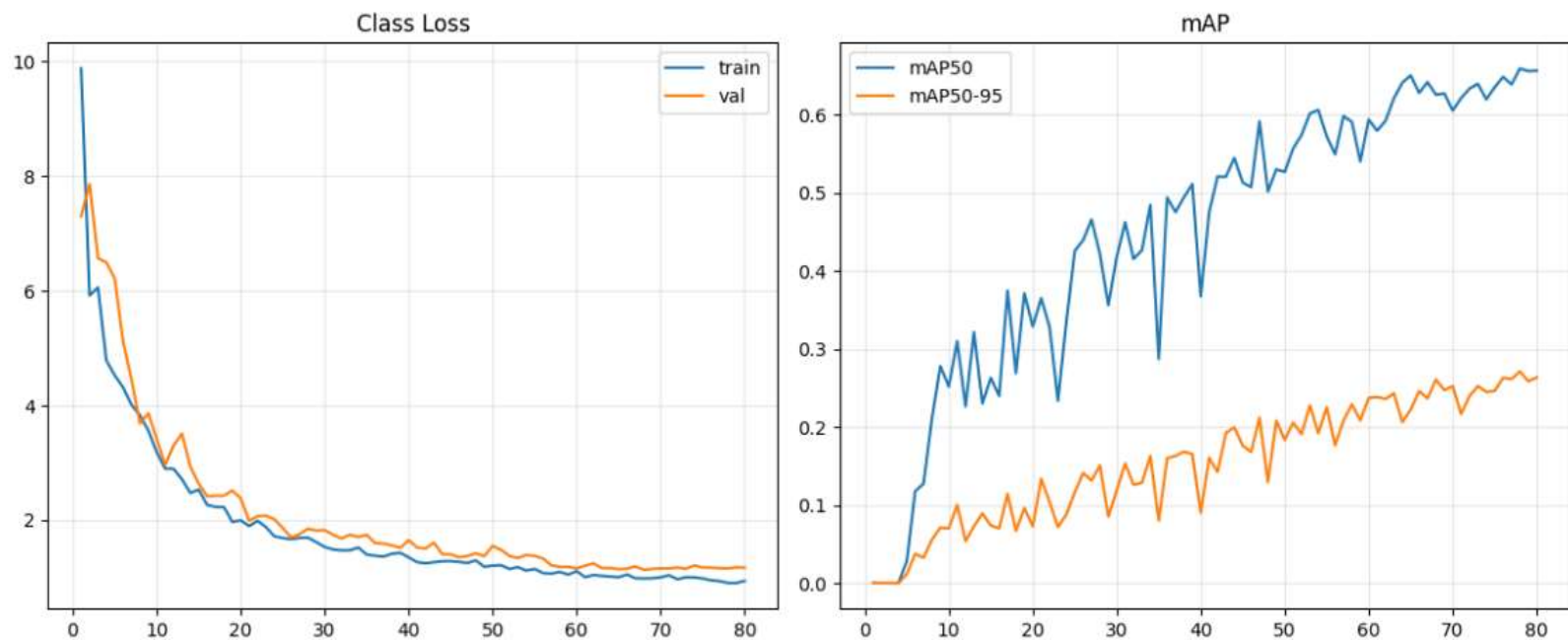
train 0.0 & val 0.0 & 균형 안 맞춤



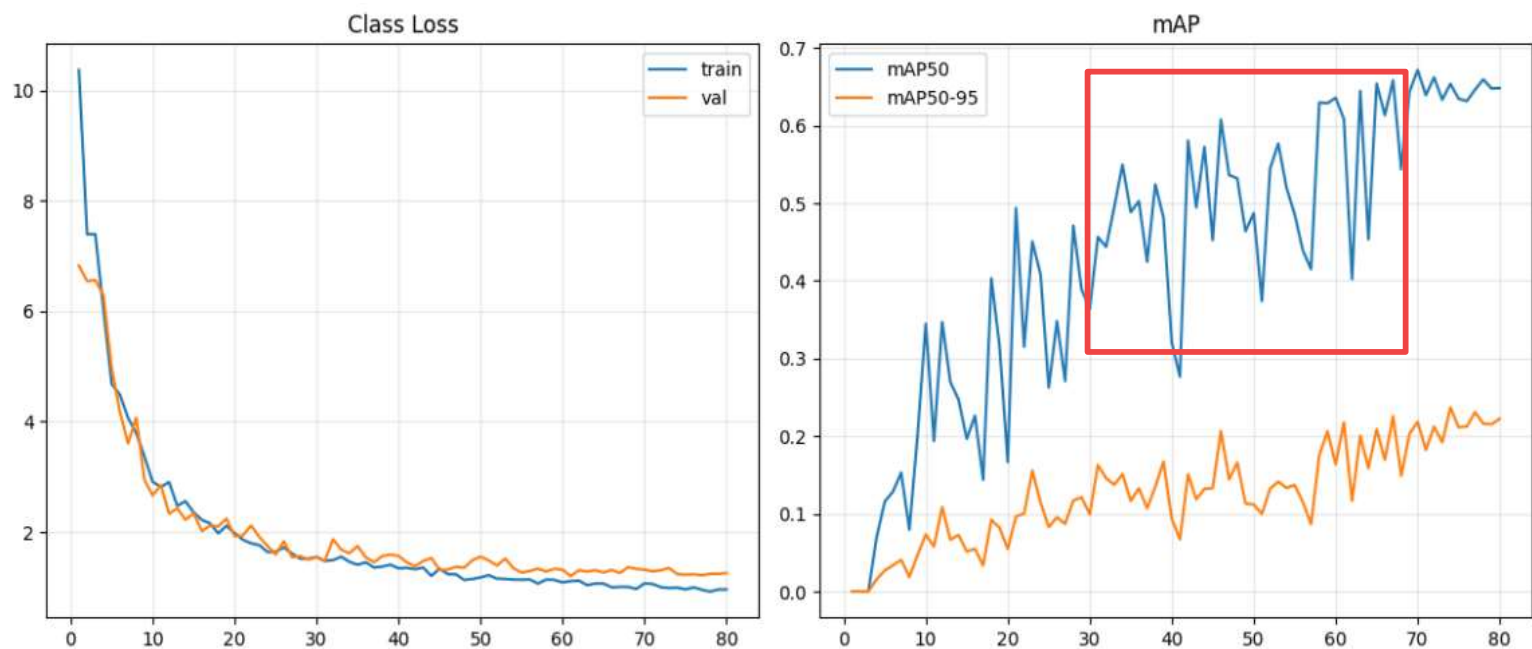
train 0.2 & val 0.0 & 균형 안 맞춤



train 0.0 & val 0.0 & 균형 맞춤

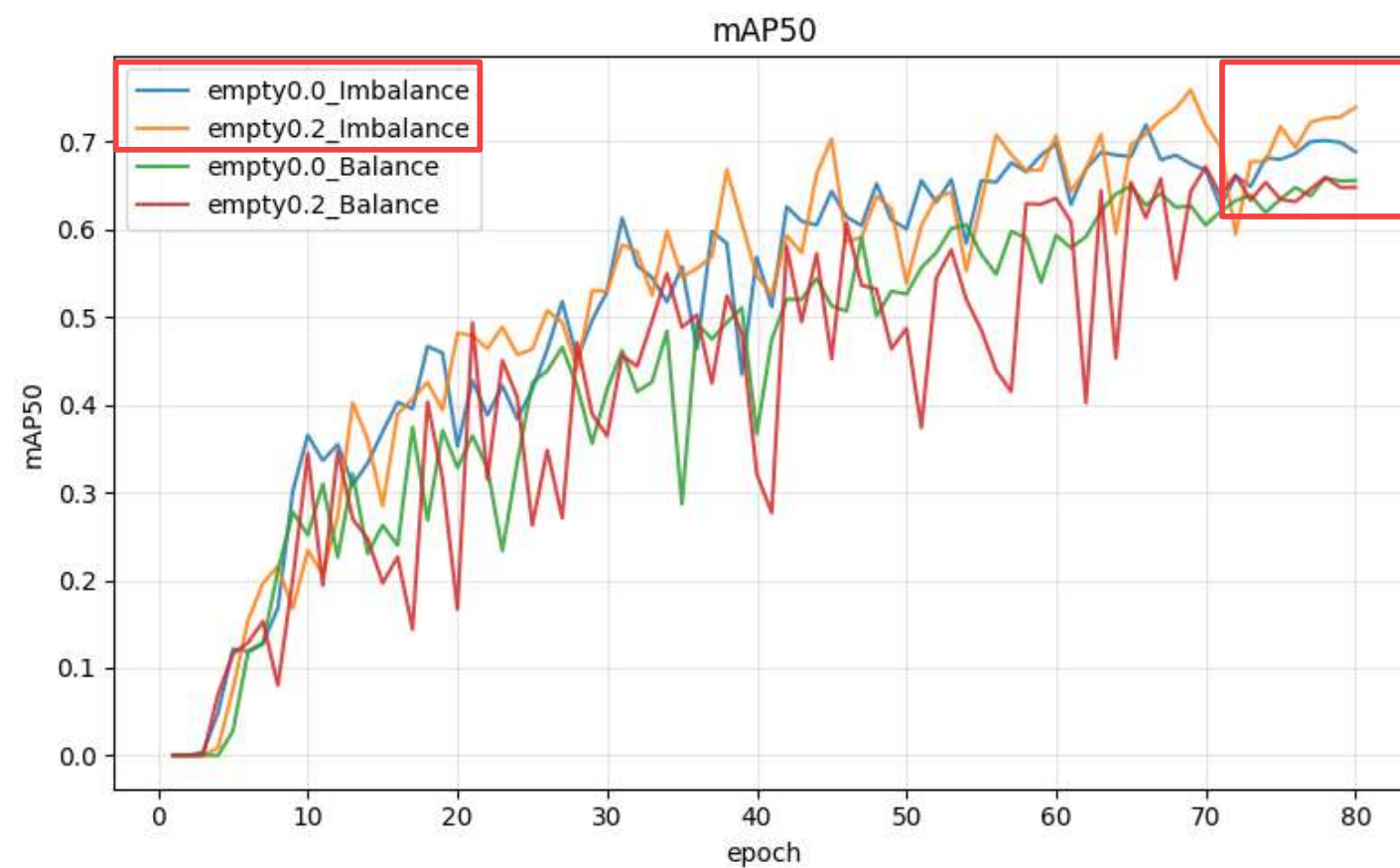


train 0.2 & val 0.0 & 균형 맞춤



Skill & Analysis

Data Processing



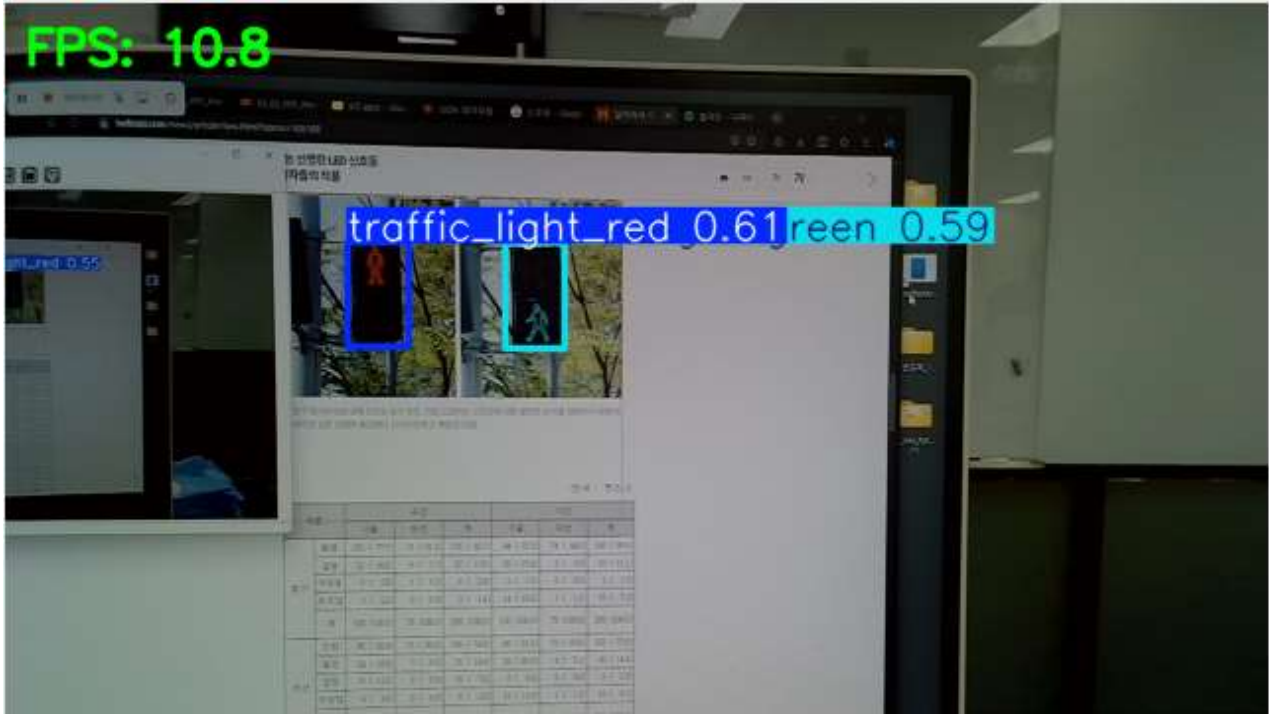
- red & green의 객체 수를 309로 동일 -> mAP50의 성능이 떨어짐
- train에서 red, green을 제외한 다른 이미지도 학습 `empty_ratio=0.2`



- 균형을 맞춘 데이터에서는 이미지의 수가 적어 20%의 empty 이미지를 넣으면 정확도가 떨어짐

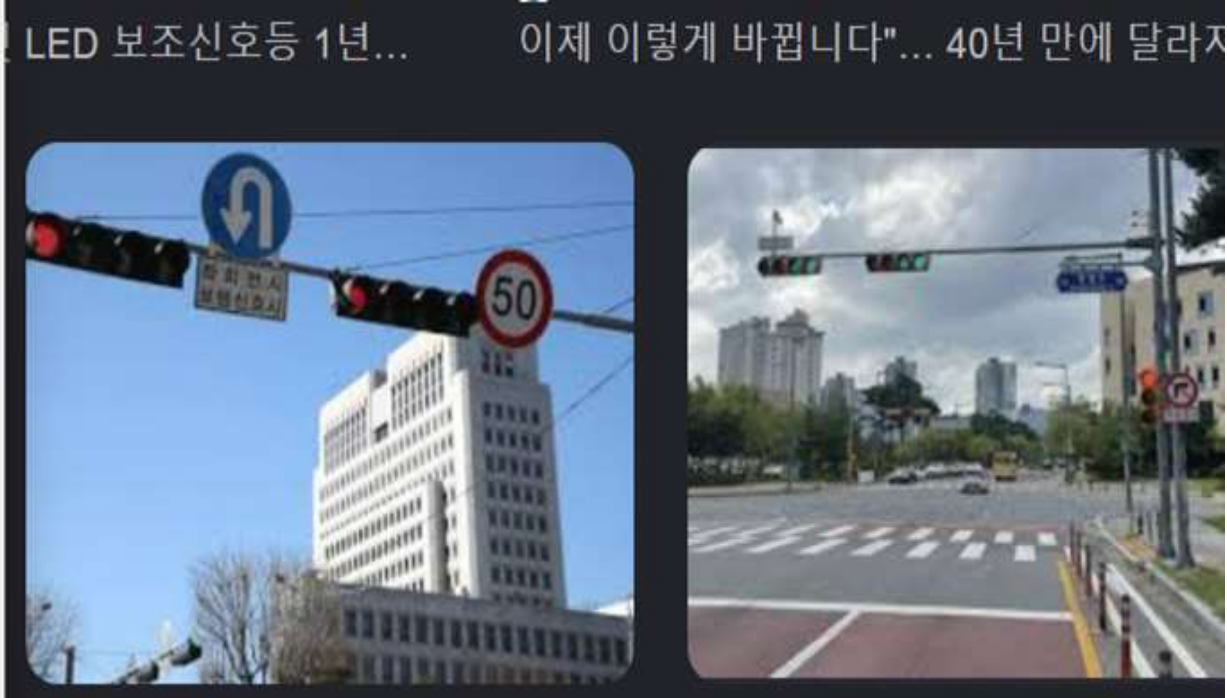
Skill & Analysis

Data Processing



연구 당시의 LED 교통신호등 설치 현장, 처음 도입하는 신호등에 대한 불안한 인식을 잠재우기 위해 대
대적인 실증 실험에 돌입했다. [사진=정학근 책임연구원]

- 보행자 신호등 red green



- 차량 신호등 red green

Skill & Analysis

Data Processing





#공공시설물 #노후화 판별 #한국형 노후시설물 데이터셋

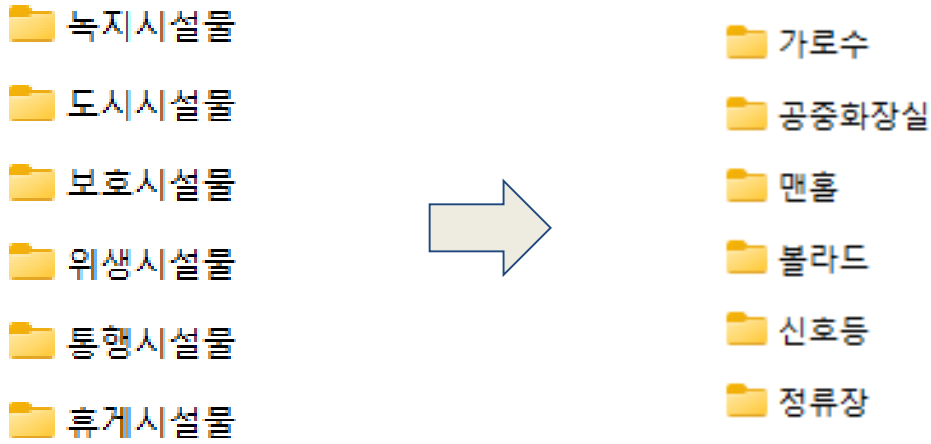
노후 시설물 이미지

분야 재난안전환경 유형 이미지

구축년도 : 2020 갱신년월 : 2021-11 조회수 : 14,010 다운로드 : 740 용량 : 2.59 TB

다운로드

↓ 샘플(경량) 데이터 ?



- 데이터의 종류가 너무 많아 class를 6개로 줄임

가로수, ... 속성

일반 사용자 지정

파일 1,200, 폴더 6

종류: 파일 폴더 형식의 모든 파일

위치: D:\노후 시설물 이미지\data의 모든 파일

크기: 2.86GB (3,075,290,342 바이트)

디스크 할당 크기: 2.86GB (3,078,565,888 바이트)

	701_10_b48de8c9-12c8-4231-b859-f7...	2021-09-16 오전 10:22	JPEG 파일	2,816KB
	701_10_b48de8c9-12c8-4231-b859-f7...	2026-06-28 오후 6:48	JSON 원본 파일	2KB
	701_10_b962b535-c72f-4f4f-a6ae-e09...	2021-09-16 오전 10:22	JPEG 파일	1,780KB
	701_10_b962b535-c72f-4f4f-a6ae-e09...	2021-10-29 오후 1:27	JSON 원본 파일	2KB
	701_10_ba87b169-eaf9-4907-847d-b9...	2021-09-03 오후 3:34	JPEG 파일	8,916KB
	701_10_ba87b169-eaf9-4907-847d-b9...	2021-08-30 오후 2:53	JSON 원본 파일	1KB
	701_10_ba89ec25-a15a-40df-9425-59a...	2021-09-03 오후 3:34	JPEG 파일	5,459KB
	701_10_ba89ec25-a15a-40df-9425-59a...	2021-08-30 오후 2:53	JSON 원본 파일	1KB
	701_10_ba187de7-0e91-4027-9ef2-ec9...	2021-09-03 오후 4:05	JPEG 파일	1,914KB
	701_10_ba187de7-0e91-4027-9ef2-ec9...	2021-08-30 오후 2:53	JSON 원본 파일	1KB

- 각 class 별로 데이터 100개, JSON 100개 -> 총 1,200개

Skill & Analysis

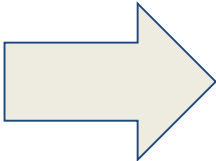
Data Processing



```
CLASSES = [  
    'StreetTreeCover',  
    'SignalPole',  
    'PublicToilet',  
    'Manhole',  
    'Bollard',  
    'StationShelter',  
]
```

이미지 파일 개수(json 제외): 600
이미지 이동: 120개, 라벨 이동: 120개
val 폴더 최종 이미지 수: 120

- class : 6개
- 총 이미지 : 600
- val : 120개



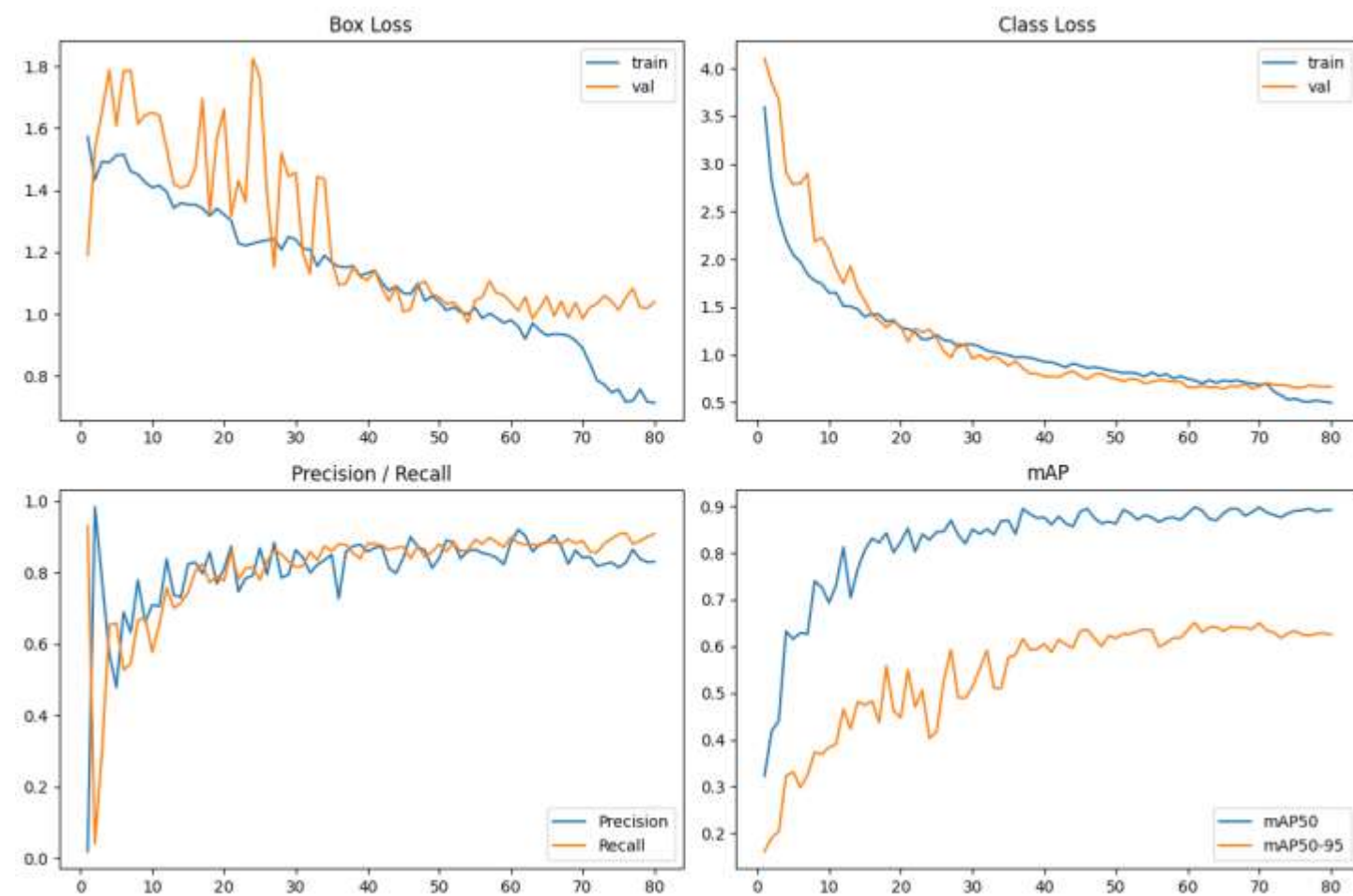
```
w 최소/최대/평균: 0.0644 / 1.0000 / 0.5232  
h 최소/최대/평균: 0.1141 / 0.9967 / 0.5208
```

- 박스의 크기가 커서 안정적으로 학습이 됨.

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
73/80	2.92G	0.7732	0.5756	1.224	17	640: 100%	30/30	2.5s/it	1:16
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	4/4 3.4it/s 1.2s
	all	120	130	0.824	0.882	0.876	0.617		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
74/80	2.92G	0.7469	0.5284	1.192	17	640: 100%	30/30	2.5s/it	1:15
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	4/4 2.4it/s 1.7s
	all	120	130	0.829	0.895	0.885	0.628		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
75/80	2.92G	0.7568	0.5411	1.21	17	640: 100%	30/30	2.5s/it	1:15
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	4/4 3.2it/s 1.3s
	all	120	130	0.815	0.907	0.89	0.633		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
76/80	2.92G	0.7169	0.5117	1.173	22	640: 100%	30/30	2.4s/it	1:12
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	4/4 3.4it/s 1.2s
	all	120	130	0.827	0.912	0.891	0.625		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
77/80	2.92G	0.7208	0.5084	1.179	17	640: 100%	30/30	2.4s/it	1:12
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	4/4 3.2it/s 1.3s
	all	120	130	0.865	0.88	0.895	0.623		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
78/80	2.92G	0.7577	0.5221	1.194	18	640: 100%	30/30	2.6s/it	1:17
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	4/4 2.8it/s 1.4s
	all	120	130	0.838	0.889	0.888	0.627		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
79/80	2.92G	0.717	0.507	1.185	18	640: 100%	30/30	2.4s/it	1:13
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	4/4 2.9it/s 1.4s
	all	120	130	0.83	0.9	0.891	0.628		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
80/80	2.92G	0.7139	0.4965	1.172	19	640: 100%	30/30	2.5s/it	1:15
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	4/4 1.8it/s 2.2s
	all	120	130	0.831	0.909	0.892	0.625		

Skill & Analysis

Data Processing



도로 시설물 6-class 학습 결과 및 추론

- 학습 곡선에서 mAP가 안정적으로 수렴했으며, 실제 추론에서 Bollard 등 시설물을 높은 신뢰도로 검출

Skill & Analysis

Data Processing



What is COCO?



COCO is a large-scale object detection, segmentation, and captioning dataset. COCO has several features:

- ✔ Object segmentation
- ✔ Recognition in context
- ✔ Superpixel stuff segmentation
- ✔ 330K images (>200K labeled)
- ✔ 1.5 million object instances
- ✔ 80 object categories
- ✔ 91 stuff categories
- ✔ 5 captions per image
- ✔ 250,000 people with keypoints

```
from ultralytics import YOLO
import os
from google.colab import files

coco_model = YOLO('yolo11n.pt')

coco_model.export(format='onnx',
                  imgsz=320,
                  opset=18,
                  simplify=True)

onnx_coco = 'rps_yolo11n_coco.onnx'
os.rename('yolo11n.onnx', onnx_coco)

save_file = f'{save_dir}/{onnx_coco}'
os.makedirs(os.path.dirname(save_file), exist_ok=True)
import shutil
shutil.copy(onnx_coco, save_file)

files.download(onnx_coco)
```

이미 포함된 클래스

보행 장애물(사람·자전거·차량·오토바이)이 COCO 80개 클래스에 이미 포함되어 별도 라벨링·학습이 필요 없음

역할 분리 설계

COCO는 사람·차량 같은 동적 장애물을, 자체 학습 모델은 신호 등·볼라드 등 정적 시설물을 담당하도록 역할을 분리

경량 모델 선택

실시간 추론을 위해 YOLO11n(nano)을 채택, ONNX·TensorRT 변환을 거쳐 Jetson Orin에 배포

Skill & Analysis

Data Processing



```
custom_model_1 = YOLO("traffic_light_balanced_v3.engine", task="detect")

coco_model = YOLO("rps_yolo11n_coco.engine", task="detect")

custom_model_2 = YOLO("obstacle_v1.engine", task="detect")

custom_model_3 = YOLO("navigation-1_yolo11n_6402.engine", task="detect")
```

```
results_custom = custom_model_1(frame, imgsz=640, conf=0.5, iou=0.45, device=0, verbose=False)
results_custom[0].names = custom_names_1

results_coco = coco_model(frame, imgsz=320, conf=0.5, iou=0.45, device=0, verbose=False, classes = [0, 1, 2, 3, 5, 7, 11, 13, 16])
results_coco[0].names = coco_names

results_custom_2 = custom_model_2(frame, imgsz=640, conf=0.5, iou=0.45, device=0, verbose=False, classes = [3, 4, 5])
results_custom_2[0].names = custom_names_2

results_custom_3 = custom_model_3(frame, imgsz=640, conf=0.5, iou=0.45, device=0, verbose=False, classes = [2, 3, 4, 5, 6])
results_custom_3[0].names = custom_names_3
```

다중 YOLO 모델 통합

- 신호등·COCO·시설물·도로요소를 담당하는 4개의 YOLO 모델을 결합
- 각 모델의 클래스 ID를 통합 라벨로 매핑하여 하나의 인식 파이프라인으로 구성
- class를 list로 받아 특정 class만 선택하여 model을 구성함.

Skill & Analysis

USB Speaker (전체 동작 흐름)



Skill & Analysis

USB Speaker (실시간 공간 인지 알고리즘)



■ 실시간 공간 인지 연산 코드

```
direction_groups = {"왼쪽에": set(), "정면에": set(), "오른쪽에": set()}
for det in all_detections:
    class_name = det['class_name']
    x1, _, x2, _ = det['box']
    x_center = (x1 + x2) / 2

    if x_center < CAMW / 3:
        direction = "왼쪽에"
    elif x_center > (CAMW * 2) / 3:
        direction = "오른쪽에"
    else:
        direction = "정면에"
    direction_groups[direction].add(class_name)
```

■ 객체 탐지 및 실시간 3분할 매핑



3분할 공간 알고리즘 핵심 원리

YOLO 모델이 추출한 객체의 바운딩 박스 중심 좌표(X축)를 계산하여, 전체 화면 영역을 3등분한 구역 중 어느 곳에 위치하는지 실시간 판별합니다.

Skill & Analysis

USB Speaker (비동기 큐 기반 실시간성 최적화)



```
audio_queue = queue.Queue()
spoken_segments = []
for direction, items in direction_groups.items():
    if items:
        items_str = ", ".join(list(items))
        spoken_segments.append(f"{direction} {items_str}")

current_spoken_text = ". ".join(spoken_segments)

if current_spoken_text:
    time_elapsed = current_time - last_spoken_time
    is_new_situation = (current_spoken_text != last_spoken_text)

    if is_new_situation or (time_elapsed > ALERT_COOLDOWN):

        while not audio_queue.empty():
            try:
                audio_queue.get_nowait()
            except queue.Empty:
                break

        print(f"[안내 발송] {current_spoken_text}")
        audio_queue.put(current_spoken_text)

        last_spoken_text = current_spoken_text
        last_spoken_time = current_time
```

중복 안내 방지 및 실시간성 필터링

- 중복 안내 방지 및 안전성 확보
- 시야 내 새로운 위험 상황 발생 시 즉각 안내
- 동일 상황은 5초 경과 후 재안내하여 사용자 피로도 최소화

오디오 출력 지연 방지 (Queue 초기화)

- 음성 지연을 막기 위해 큐에 누적된 과거 데이터 강제 삭제
- 항상 현재 상황이 스피커로 출력되도록 보장

최신 정보 업데이트 및 대기열 삽입

- 최신 상태 갱신 및 출력 대기열 삽입
- 과거 데이터를 비운 큐에 가장 최신화된 시각 정보 전달
- 현재 안내한 텍스트와 시간을 기록하여 다음 쿨다운 계산에 활용

Skill & Analysis

USB Speaker (리눅스 오디오 우회 및 하드웨어 다이렉트 제어)



```
def audio_worker():
    while True:
        message = audio_queue.get()
        print(f"[고음질 음성 출력 중...] {message}")

        try:
            tts = gTTS(text=message, lang='ko')
            tts.save("temp.mp3")
            os.system('mpg123 -q -a plughw:3,0 temp.mp3')

        except Exception as e:
            print(f"❌ [오디오 재생 실패]: {e}")

        finally:
            audio_queue.task_done()
```

시각장애인 맞춤형 고음질 음성 합성 (gTTS)

- 내장 기계음 모듈 제거로 전달력 개선
- 구글 클라우드 기반 gTTS를 활용하여 자연스럽고 명확한 한국어 안내 음성 실시간 생성

리눅스 오디오 믹서 우회 및 하드웨어 다이렉트 제어

- OS 믹서를 거치지 않고 USB 스피커 물리 주소로 음성 데이터 직접 전송
- 소프트웨어 간섭을 최소화하여 딜레이 없는 완벽한 실시간 출력 확보

예외 처리 및 스레드 동기화

- 오디오 재생 중 충돌이 발생해도 메인 카메라 시스템이 다운되지 않도록 안전 장치 확보
- 재생이 완전히 끝나면 대기열에 완료 신호를 보내 시스템 안정성 유지 및 다음 안내 준비

Skill & Analysis

Bluetooth Speaker



블루투스 스피커도 동일한 코드 적용



Skill & Analysis

USB Speaker (Trouble shooting)

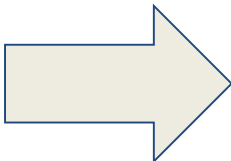


이슈: USB 스피커 묵음 현상

- 비전 인식(YOLO) 코드는 정상적으로 동작하나, 스피커에서 아무런 안내 음성이 출력되지 않는 문제 발생

원인 : OS 믹서 충돌

- 젯슨 보드(리눅스) 환경에서 파이썬 오디오 라이브러리가 USB 스피커의 출력 경로를 찾지 못함
- 기본 사운드 서버(pulseaudio)의 복잡한 믹서 레이어에서 오디오 신호가 증발하는 현상 확인



```
card 3: UACDemoV10 [UACDemoV1.0], device 0: USB Audio [USB Audio]
Subdevices: 0/1
Subdevice #0: subdevice #0
```

[Step 1] 하드웨어 주소 확인

터미널(**aplay -l**)을 통해 스피커의 물리 주소가 **card 3**임을 확인

```
echo -e "pcm.!default {\n type hw\n card 3\n}..." > ~/.asoundrc
```

[Step 2] 기본 오디오 출력 고정

리눅스 사운드 설정 파일(**.asoundrc**)을 직접 수정하여 시스템 기본 오디오 출력을 **3번 카드**로 강제 고정

```
os.system('mpg123 -q -a plughw:3,0 temp.mp3')
```

[Step 3] 초경량 플레이어 우회 제어

파이썬 라이브러리의 불안정성을 완벽히 제거하기 위해, 고음질 음성 합성 후 초경량 플레이어 채택

소프트웨어 믹서를 거치지 않고 스피커 하드웨어 포트에 데이터를 직결하는 우회 옵션 적용

Skill & Analysis

Ollama (gemma3:4b)



- LLM 사용 목적
-> 객체 탐지 후 보행자의 상황에 맞는 안내 문장으로 출력
- 카메라부터 소리 출력까지의 과정
객체 탐지 -> 자연어 처리 -> 음성 합성(TTS)



Skill & Analysis

Ollama (gemma3:4b) -자연어 처리 핵심 제어 기술



1. PROMPT 제어 기술

- 도입: 정확한 정보 전달
- 과정: 안내사항을 20자 이내로 설정
- 결과: 불필요한 부연 설명을 제거하여 빠른 정보 전달

2. HISTORY 관리 기술

- 도입: 메모리 누적에 따른 성능 저하 방지
- 과정: 대화 기록을 담고 있는 배열의 크기가 6을 초과하면 오래된 데이터 삭제
- 결과: 가장 최근 4개의 데이터만 보존하여 리소스 줄임 -> 연산 속도 유지

3. Multi-Threading 기술 도입

- 도입: 객체 탐지 시간, 자연어 처리 시간 다름
- 문제: OLLAMA가 실행되는 동안 객체 인식 멈춤
- 과정: 메인 쓰레드, OLLAMA Thread 분리
- 결과: OLLAMA가 상황을 요약하여 스피커를 출력하는 동안 YOLO 모델은 독립적인 객체 탐지 수행

Skill & Analysis

Ollama (qwen2:0.b)-model 변경



[Ollama 분석 시작]: 정면에 횡단보도

[성능 분석] 속도: 3.65 tokens/s | 생성된 토큰: 14

[스피커 출력]: 횡단보도가 있어, 갑자기 사람이 나올 수 있습니다.

[Ollama 분석 시작]: 오른쪽에 횡단보도

[성능 분석] 속도: 3.55 tokens/s | 생성된 토큰: 17

[스피커 출력]: 오른쪽 횡단보도가 있으니 주의를 기울이세요.

[Ollama 분석 시작]: 정면에 횡단보도

[스피커 출력]: 초록불입니다. 건너셔도 좋습니다.

[성능 분석] 속도: 3.55 tokens/s | 생성된 토큰: 14

[스피커 출력]: 횡단보도가 있어, 갑자기 사람이 나올 수 있습니다.

[Ollama 분석 시작]: 정면에 횡단보도

[스피커 출력]: 초록불입니다. 건너셔도 좋습니다.

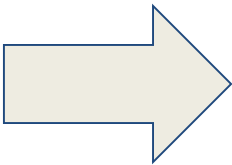
[성능 분석] 속도: 3.81 tokens/s | 생성된 토큰: 16

[스피커 출력]: 횡단보도가 있으니, 건너는 사람을 주의하세요.

[Ollama 분석 시작]: 정면에 횡단보도

[성능 분석] 속도: 3.86 tokens/s | 생성된 토큰: 15

[스피커 출력]: 횡단보도가 앞에 있어, 조심해서 지나가세요.



[Ollama 분석 시작]: 정면에 횡단보도

[성능 분석] 속도: 17.80 tokens/s | 생성된 토큰: 16

[스피커 출력]: "현재 상태는 정면에 횡단보도입니다."

[Ollama 분석 시작]: 정면에 횡단보도

[성능 분석] 속도: 13.73 tokens/s | 생성된 토큰: 18

[스피커 출력]: "현재 상태는 정면에 횡단보도를 가지고 있습니다."

[Ollama 분석 시작]: 정면에 횡단보도

[성능 분석] 속도: 17.35 tokens/s | 생성된 토큰: 17

[Ollama 분석 시작]: 왼쪽에 횡단보도

[스피커 출력]: "현재 상태는 정면에 횡단보도가 있습니다."

[성능 분석] 속도: 15.10 tokens/s | 생성된 토큰: 18

[스피커 출력]: "현재 상태는 왼쪽에 횡단보도가 있습니다."

[스피커 출력]: 초록불입니다. 건너셔도 좋습니다.

[Ollama 분석 시작]: 정면에 올라가는계단

[스피커 출력]: 주의. 내려가는 계단이 있습니다.

[성능 분석] 속도: 17.79 tokens/s | 생성된 토큰: 18

[스피커 출력]: "현재 상태는 정면에 올라가는 계단이 있습니다."

측정항목	Gemma3:4b(전)	Qwen2:0.5b(후)	개선 폭
평균 생성 속도	약 3.64 tokens/s	약 18.7 tokens/s	약 5.1배 향상
평균 생성 토큰	약 14.3개	약 17.5개	-
평균 지연 시간	약 3.93초	약 0.94초	약 2.99초 단축

Future Plan





FUTURE PLANS

01

3D 거리 지각(Depth) 도입

현재 한계

거리 정보 부재

화면상 객체의 종류와 위치만 파악할 뿐 물리적 거리 파악이 불가능하여, 충돌 위험의 시급성 판단 불가

후속 개발 방향

- 픽셀 비율 및 화면 내 원근법을 활용한 3단계(매우 가까움/멀음 등) 거리 추정 알고리즘 적용
- 객체와의 거리에 따라 음성 알람의 시급성을 차등 적용하여 보행자의 충돌 대비 시간 확보

02

주체자 이동 보상(Ego-Motion 및 객체 추적)

현재 한계

정적 객체 오탐지

보행자가 앞으로 이동함에 따라 주차된 차량 등 정지 객체가 커지는 현상을 '다가오는 위협'으로 잘못 인식

후속 개발 방향

- Bounding Box의 확장률(상대 속도)을 분석하여, 정적 장애물과 동적 장애물(주행 중인 차)을 분리하고 실제 위협만 타기팅

03

하이브리드 상황 인지 및 안내 최적화

현재 한계

정보 과부하 및 획일화된 알림

시야에 들어오는 모든 사물을 안내하여 보행 동선과 직결된 핵심 위험 상황의 전달력 저하

후속 개발 방향

- 보행 진행 방향(화면 중앙부) 중심의 안전 구역(Safe Zone) 설정 및 외곽의 불필요한 객체 알림 소거
- 연속 프레임의 객체 흐름을 분석하여 사용자의 이동 방향을 추론하는 알고리즘 도입

Retrospective





Retrospective

김지홍

- 기존에 경험했던 CNN과 달리, YOLO 모델이 실시간으로 Classification 및 Detection하는 과정이 흥미로웠음.
외부 클라우드 없이 Jetson 보드 내부에서 로컬 LLM이 스스로 상황을 판단하고 문장을 만들도록 유도한 것이 재밌었음.

최은수

- 데이터를 정제하는 것이 굉장히 어렵다는 것을 알게 되었음. 항상 좋아지는 것이 있으면 그에 반해서 안 좋아지는 것이 존재하여 데이터의 특성과 유형에 따라서 선택과 집중이 필요하다는 것을 깨달음.

조준호

- 단순한 AI 기술 구현을 넘어, 시각장애인이라는 실제 사용자의 입장에서 UX를 깊이 고민해 본 의미 있는 경험이었음.
AI 소프트웨어도 최적화된 하드웨어 제어와 연결되지 않으면 성능을 낼 수 없다는 시스템 통합의 중요성을 체감하였음.

윤수현

- 이번 프로젝트를 통해 배운 점은 '최고의 AI'가 항상 '최적의 AI'는 아니라는 것임. 아무리 똑똑하고 무거운 모델이라도 보행자에게 제때 위험을 알리지 못하면 의미가 없다는 것을 인지함. 이를 통해 서비스의 요구사항에 맞는 적정 기술을 선택하는 안목을 기를 수 있었음.



THANK YOU

8팀 (눈이돼조)